```
    on: (event, callback) ->
    # do nothing

  tracking = new Tracking options, http
  tracking.start ->
  fred.visit '/page/1'
  assert.equal tracking.total(), 1
```

Perhaps you don't really need an `http` double at all. Instead, you can create a double of a method that *uses* it. Replace the `start` method with a double:

```
fact 'the tracking application tracks a user mouse click', ->
  tracking = new Tracking options, http
  tracking.start = (callback) ->
    callback()

  fred.visit '/page/1'
  assert.equal tracking.total(), 1
```

This example oversimplifies the scenario, but you can clearly see that instead of a double for an object, you essentially have a double for a function. This is often referred to as a *method stub*, but you can think of it like any other double.

10.4.3. Expect it

Suppose you're testing a program that talks to a database via this `http` object. You don't really care about the response you get back asynchronously from the database. What you *do* care about is that you made the correct call to it. If the correct call was made, you trust the database to do the right thing. After all, a database that doesn't store things when you tell it to is probably not a database worth having.

Consider the `http` module; you might *expect* a particular method to be called with specific parameters. A simple way to test that the call happened is to remember it in the test, using a variable: